(*
Variables

We now move from expressions to formulas, i.e., expressions that contain *variables*. As the name implies, a variable is a 0-ary symbol can be given different values, in contrast with constants in a signature Σ, whose values are fixed, once we fix a Σ-algebra $\mathcal{A}$. Assume a (denumerable) set $\mathcal{X}$ of variables, disjoint from symbols in Σ, i.e., $\mathcal{X} \cap \Sigma = \emptyset$ *)
(*
Scaling back from the example toy language that we have already developed, let us illustrate the ideas of Tree algebras, and Σ-homomorphisms:
Let us represent variables using the OCaml type `string` and extend the encoding of the abstract syntax for expressions in the data type `exp`:
*)

```
type exp = Num of int
         | V of string
         | Plus of exp * exp
         | Times of exp * exp;;
```

(* Note that we have a new family of base cases: variables, which are denoted using the parameterised constructor `V(_)` which takes arguments of type `string`.
*)
(* The corresponding Concrete Syntax can also be extended

```
E —> T
E —> T + E
T —> F
T —> F * T
F —> n
F —> v      // a family of variables
F —> ( E )
```

*)
(*
We now extend the syntactic functions on tree-structured expressions *)

(* `size`, as before, returns the number of nodes in the tree, with a variable counted as a single node. This is an *extension* to deal with variables of a Σ-homomorphism from `exp`, i.e., $\mathbf{Tree}_\Sigma$ to the integers $\mathbb{Z}$, where all leaves are mapped to 1, and constructors `Plus` and `Times` are both interpreted as the function $f(x_1, x_2) \in \mathbb{Z}^2 \to \mathbb{Z} = 1 + x_1 + x_2$.
*)

```
let rec size t = match t with
    Num _ -> 1
  | V _ -> 1
  | Plus(t1, t2) -> 1 + (size t1) + (size t2)
  | Times(t1, t2) -> 1 + (size t1) + (size t2)
;;
```

(* `ht`, also as before, returns one less than the number of levels in the tree, i.e., length of the longest path from root, with variables being treated as leaf nodes. This is an *extension* of the previous definition of `ht` to deal with variables of a ∑-homomorphism from `exp`, i.e., **Tree$_\Sigma$** to the integers $\mathbb{Z}$, where all leaves are mapped to 1, and constructors `Plus` and `Times` are both interpreted as the function $f(x_1, x_2) \in \mathbb{Z}^2 \rightarrow \mathbb{Z} = 1 + (max\ x_1\ x_2)$.
*)

```
let rec ht t = match t with
    Num _ -> 0
  | V _ -> 0
  | Plus(t1, t2) -> 1 + (max (ht t1) (ht t2))
  | Times(t1, t2) -> 1 + (max (ht t1) (ht t2))
;;
```

(* As mentioned before, both `size` and `ht` are good measures for performing induction on trees. *)

(* We now define a syntactic function on `exp` called `vars` which returns the *list* of variables in the tree. (We will later see that we really want a function returning the *set* of variables in a syntax tree. If we represent sets as lists, then the difference goes away; of course, we must avoid duplicates and not care about the relative order in which elements appear). *)

```
let rec vars t = match t with
    Num _ ->   [ ]
  | V x ->   [x]
  | Plus(t1, t2) -> (vars t1) @ (vars t2)
  | Times(t1, t2) -> (vars t1) @ (vars t2)
;;
```

(* We now extend the *standard semantics* — the definitional interpreter of a language of expressions which include variables. The main question to answer is "what is the value of an expression that contains variables?" The way to approach this question is to pose a counter-question, asking what value each variable takes. So `eval` now takes an extra argument, which we call `rho`, that maps each variable to a value. For the most part, except the new base cases, argument `rho` is carried along as extra baggage in each call. But it cannot be left out. (Why not?) Note that `rho` is a function from `string` to `int` ! *)

```
let rec eval t rho = match t with
    Num n ->  n
  | V x -> rho x
  | Plus(t1, t2) -> (eval t1 rho) + (eval t2 rho)
  | Times(t1, t2) ->  (eval t1 rho) * (eval t2 rho)
;;
```

(* We now extend the `compile` function. First we introduce an opcode `LOOKUP` to look up the value associated with a given variable.
*)

```
type opcode = LD of int
            | LOOKUP of string //look up variable's value
            | PLUS
            | TIMES
;;
```

(* The compiler as before, is a post-order traversal, i.e., a tree-recursive function, with a straightforward new base case. *)

```
let rec compile t = match t with
    Num n ->  [LD (n)]
  | V x -> [ LOOKUP(x) ]
  | Plus(t1, t2) -> (compile t1) @ (compile t2) @ [PLUS]
  | Times(t1, t2) ->  (compile t1) @ (compile t2) @ [TIMES]
;;
```

(* The stack machine is again written as a tail recursive function driven by the first op-code and the state of the stack, and a new component, a table `env` (environment, "gamma"), which maps variables to their corresponding *answers*.
*)

```
let rec stkmc env s code = match s, code with
   a::s, [ ] ->  a
  | s, (LD(n)) :: code' ->  stkmc ((Num n) :: s) code'
  | s, (LOOKUP(x))::code' -> let a= (env x)
                             in  stkmc env (a::s)  code'
  | (Num n1) :: (Num n2) :: s', PLUS :: code' ->
                          let n = n1 + n2
                           in  stkmc env ((Num n) :: s') code'
  | (Num n1) :: (Num n2) :: s', TIMES :: code' ->
                          let n = n1 * n2
                           in stkmc env ((Num n) :: s') code'
  | _ -> raise (Stuck(env, s, code))
;;
```

(* Let us now abstract this treatment to arbitrary abstract syntax trees over arbitrary signatures ∑. We do so by defining a set $Tree_{\Sigma}(\mathcal{X})$.

Definition: Suppose ∑ is a given signature.
Suppose $\mathcal{X}$ is a (denumerable) set of variables.
The set $Tree_{\Sigma}(\mathcal{X})$ is inductively defined as follows:

Base cases:
- for each 0-ary symbol c in ∑, a labelled node $\bullet c$ is in $Tree_{\Sigma}(\mathcal{X})$
- for each variable x in $\mathcal{X}$, *a labelled node* $\bullet x$ is in $Tree_{\Sigma}(\mathcal{X})$.

Induction cases: for each $k > 0$,
for each k-ary symbol f in ∑,

for *any* trees $t_1, \ldots, t_k$ in $Tree_\Sigma(\mathcal{X})$,
the tree consisting of a labelled node $\bullet f$ at the root
with k trees $t_1, \ldots, t_k$ ordered as subtrees below the root node
is in $Tree_\Sigma(\mathcal{X})$.

The algebra of abstract syntax trees

$\mathbf{Tree}_\Sigma(\mathcal{X})$ is the Σ-algebra with carrier set $Tree_\Sigma(\mathcal{X})$, where (as before)
- every 0-ary symbol c in Σ is interpreted as the labelled node $\bullet c$, in $Tree_\Sigma(\mathcal{X})$,
- every k-ary symbol ($k > 0$) f in Σ is interpreted as the tree-forming function
that takes k trees $t_1, \ldots, t_k$ from $Tree_\Sigma(\mathcal{X})$, and creates the tree

$\bullet f$
/ \
$t_1 \ldots t_k$

by sticking $t_1, \ldots, t_k$ as the subtrees below a new root node labelled $\bullet f$. *)
(* Observe $\mathcal{X}$ is in 1-1 correspondence with a subset $\bullet\mathcal{X}$ of $Tree_\Sigma(\mathcal{X})$.
Note also that $Tree_\Sigma(\mathcal{X})$ contains $\bullet\mathcal{X} \cup Tree_\Sigma$.

Definition (B-valuation). A function $\rho : \mathcal{X} \to B$ that maps variables to values in a carrier set B is called a B-valuation.

Definition (Function extension) Let $A_1 \subseteq A_2$ and let $f_1 : A_1 \to B$. Function $f_2 : A_2 \to B$ *extends* f_1 (to domain A_2) if for all $a \in A_1$, $f_2(a) = f_1(a)$.

Definition (Function extension on valuations)
A function $h: Tree_\Sigma(\mathcal{X}) \to B$ extends function $\rho : \mathcal{X} \to B$ if
for all $x \in \mathcal{X}$, $h(\bullet x) = \rho(x)$

Unique Homomorphic Extension Theorem

For any signature Σ, and
any Σ-algebra, $\mathcal{B} = \langle B, \ldots \rangle$,
and any B-valuation $\rho : \mathcal{X} \to B$,
there is a unique Σ-homomorphic extension of $\rho : \mathcal{X} \to B$, written
$\hat{\rho} : Tree_\Sigma(\mathcal{X}) \to B$
from the (syntactic) Σ-algebra $\mathbf{Tree}_\Sigma(\mathcal{X})$ to the Σ-algebra $\mathcal{B}$.

Proof (First) Existence of a Σ-homomorphic extension of $\rho : \mathcal{X} \to B$.
By construction: define $\hat{\rho} : Tree_\Sigma(\mathcal{X}) \to B$ to be a Σ-homomorphism as follows:
- For each 0-ary symbol c in Σ: $\hat{\rho}(\bullet c) = c_{\mathcal{B}}$
- For each $x \in \mathcal{X}$, $\hat{\rho}(\bullet x) = \rho(x)$
- For each k-ary symbol ($k > 0$) f in Σ, and $t_1, \ldots, t_k$ in $Tree_\Sigma(\mathcal{X})$

$\hat{\rho}(\bullet f) = f_{\mathcal{B}}(\hat{\rho}(t_1), \ldots, \hat{\rho}(t_k))$
/ \
$t_1 \ldots t_k$

Uniqueness of $\hat{\rho}$

Suppose j is an extension of ρ, and $j : Tree_\Sigma(\mathcal{X}) \to B$ is a Σ-homomorphism from Σ-algebra $\mathbf{Tree}_\Sigma(\mathcal{X})$ to the Σ-algebra $\mathcal{B}$.

Proof by induction on (ht t) that for all t in $Tree_\Sigma(\mathcal{X})$, $\hat{\rho}(t) = j(t)$

Base cases (ht t) = 0.

subcase t is of the form $\bullet c$

for each 0-ary symbol c in Σ: $\hat{\rho}(\bullet c) = c_{\mathcal{B}}$ // defn of $\hat{\rho}$

$= j(\bullet c)$ // j is a Σ-homomorphism // from $\mathbf{Tree}_\Sigma(\mathcal{X})$ to $\mathcal{B}$

subcase t is of the form $\bullet x$ for some $x \in \mathcal{X}$

$\hat{\rho}(\bullet x) = \rho(x)$ // defn of $\hat{\rho}$

$= j(\bullet x)$ // j extends ρ

Induction Hypothesis:

Assume that for all t' in $Tree_\Sigma(\mathcal{X})$ such that (ht t') $\leq n$,

$\hat{\rho}(t') = j(t')$

Induction Step: Consider t in $Tree_\Sigma(\mathcal{X})$ s.t. (ht t) = $n+1$

t must be of the form

$\bullet f$

$/ \quad \backslash$

$t_1 \ldots t_k$

with (ht t_i) $\leq n$ $(1 \leq i \leq k)$

Now, by definition,

for each k-ary symbol ($k > 0$) f in Σ, and $t_1, \ldots, t_k$ in $Tree_\Sigma(\mathcal{X})$ s.t. (ht t_i) $\leq n$

$\hat{\rho}(\bullet f)$

$/ \quad \backslash$

$t_1 \ldots t_k$

$= f_{\mathcal{B}}(\hat{\rho}(t_1), \ldots, \hat{\rho}(t_k))$ // definition of $\hat{\rho}$

$= f_{\mathcal{B}}(j(t_1), \ldots, j(t_k))$ // by IH on each of $t_1 \ldots t_k$

$= j(\bullet f)$ // $j : Tree_\Sigma(\mathcal{X}) \to B$ is a Σ-homomorphism

$/ \quad \backslash$ // from Σ-algebra $\mathbf{Tree}_\Sigma(\mathcal{X})$ to the Σ-algebra $\mathcal{B}$.

$t_1 \ldots t_k$

*)

(* Only those variables that appear in a tree (term, expression) matter in its evaluation.

Lemma (Relevant Variables): Suppose Σ is an *arbitrary* signature and $\mathcal{X}$ is the set of variables.

Let t in $Tree_\Sigma(\mathcal{X})$ be an arbitrary tree (term, expression).

Let V = vars(t). // set of variables that appear in t

Consider any Σ-algebra $\mathcal{B} = \langle B, \ldots \rangle$, and let $\rho_1, \rho_2 \in \mathcal{X} \to B$ be any two B-valuations such that *for all* variables $x \in V$, $\rho_1(x) = \rho_2(x)$.

[Note: $\rho_1(y)$ and $\rho_2(y)$ may be very different for $y \notin V$.]
Then $\widehat{\rho_1}(t) = \widehat{\rho_2}(t)$. [Here $\widehat{\rho_1}$, $\widehat{\rho_2}$ are the unique Σ-homomorphic extensions of ρ_1, ρ_2 respectively, i.e., with respect to the Σ-algebra $\mathcal{B} = \langle B, \ldots \rangle$]

Proof by induction on `ht(` t `)`.
Base cases (`ht(` t `)` = 0).
subcases: t is of the form $\bullet c$ for some 0-ary symbol c in Σ.
Therefore $\widehat{\rho_1}(\bullet c) = c_{\mathcal{B}} = \widehat{\rho_2}(\bullet c)$
subcases t is of the form $\bullet x$ for some x in $\mathcal{X}$. So $x \in V$.
Therefore $\widehat{\rho_1}(\bullet x) = \rho_1(x) = \rho_2(x) = \widehat{\rho_2}(\bullet x)$
Induction Hypothesis: Assume *for all* t' in $Tree_\Sigma(\mathcal{X})$ such that `ht(` t' `)` $\leq n$,
for all $\rho'_1, \rho'_2 \in \mathcal{X} \to B$ such that
for all variables $x \in V'$ = `vars(` t' `)`, $\rho'_1(x) = \rho'_2(x)$,
that $\widehat{\rho'_1}(t') = \widehat{\rho'_2}(t')$.
Induction Step: Let t in $Tree_\Sigma(\mathcal{X})$ be such that `ht(` t `)` = n+1.
By analysis t is of the form $\bullet f$ for some symbol f of arity k in Σ

$t_1 \ldots t_k$

and trees $t_1, \ldots, t_k$ from $Tree_\Sigma(\mathcal{X})$, where `max ht(` t_i`)` = n. (1≤i≤k)

$\widehat{\rho_1}(\bullet f) = f_{\mathcal{B}}(\widehat{\rho_1}(t_1), \ldots, \widehat{\rho_1}(t_k))$ // defn of $\widehat{\rho_1}(t)$

$t_1 \ldots t_k$

$= f_{\mathcal{B}}(\widehat{\rho_2}(t_1), \ldots, \widehat{\rho_2}(t_k))$ // By IH on $t_1, \ldots, t_k$ with $\rho'_1 = \rho_1$ and $\rho'_2 = \rho_2$
// Note that V_i = `vars(` t_i `)` $\subseteq$ `vars(` t`)` = V
// so if *for all* $x \in V$, $\rho_1(x) = \rho_2(x)$,
// then *for all* $x \in V_i$, $\rho_1(x) = \rho_2(x)$ (1≤i≤k)

$= \widehat{\rho_2}(\bullet f)$

$t_1 \ldots t_k$

*)

(*
Proposition: For any t in $Tree_\Sigma(\mathcal{X})$, V = `vars(` t `)` is finite.
Corollary: t in $Tree_\Sigma(\mathcal{X})$ can evaluate to at most $|B|^{|V|}$ values.
*)